



Universidad Central de Venezuela
Facultad de Ciencias
Licenciatura en Ciencias de la Computación
Fundamentos de la Programación Paralela



Informe.

Ejercicio nBodies.

Alumno:

Br. Anthony Alvarado

Docente:

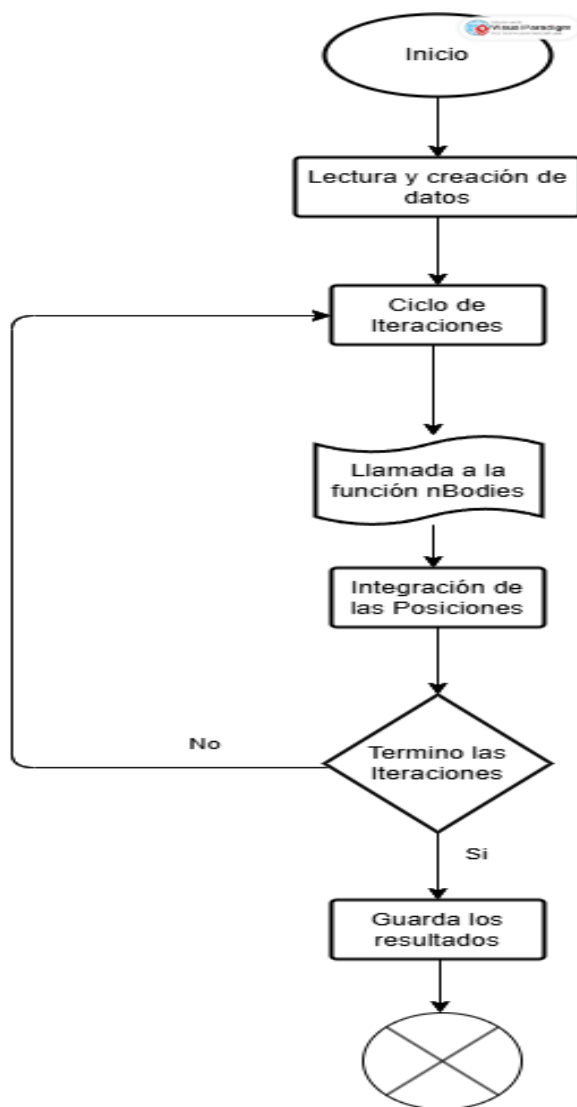
MSc. Robinson Rivas

Caracas, 21 de marzo de 2025

Problema de los nBodies.

En el código que se nos entregó se encontraba la solución para este problema, pero el mismo se realizaba de manera secuencial, lo cual a pesar de ser rápido con la cantidad de partículas dadas al aumentar su número lo más probable es que el tiempo aumentara de manera exponencial, esto debido a que dentro del código podemos encontrar una función la cual es de orden n^2 y esta además de realizar varias veces el n va a ser tan grande como la cantidad de partículas que son dadas.

El código secuencial se puede entender mediante el siguiente diagrama:



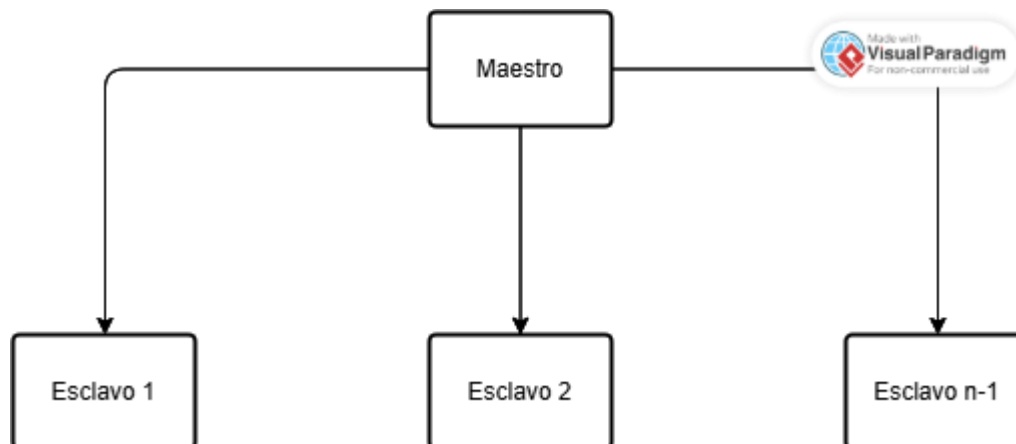
Como se puede apreciar es un código muy fácil, el problema se encuentra en “Llamada a la función nBodies” debido a que esta llama una función la cual su orden es n^2 , es decir, tiene que recorrer la cantidad de partículas 2 veces.

Posible solución:

Una posible solución es aplicar programación paralela a este código, pero no a todo, como mencione anteriormente el problema es este código está en la función llamada “nBodies”, esta función es la que podemos paralelizar para que se resuelva en menos tiempo del esperado en el secuencial. Esto se espera así debido a que el código secuencial un solo procesador tiene que revisar las n partículas, es decir, el orden sería orden n , mientras que si le aplicamos programación paralela podremos dividir el número de partículas (n) entre el número de procesadores (np) cuya solución (siempre y cuando np sea mayor a 1) va a ser mucho mejor que la secuencial.

Consideraciones al aplicar paralelismo:

Una consideración al aplicar paralelismo es la definición de roles, los cuales los vamos a dividir en un **maestro** y $n-1$ **esclavos** (siendo $n-1$ la cantidad de procesadores disponibles menos el del maestro).



Otro punto a tener en cuenta es las funciones que van a cumplir dentro del código, esto debido a que si no se realiza de manera correcta la distribución de la función de cada uno puede existir un interbloqueo o que en algunos de los procesos se genere una inanición.

Teniendo esto claro vamos a definir las funciones de cada uno:

Para el **Maestro**:

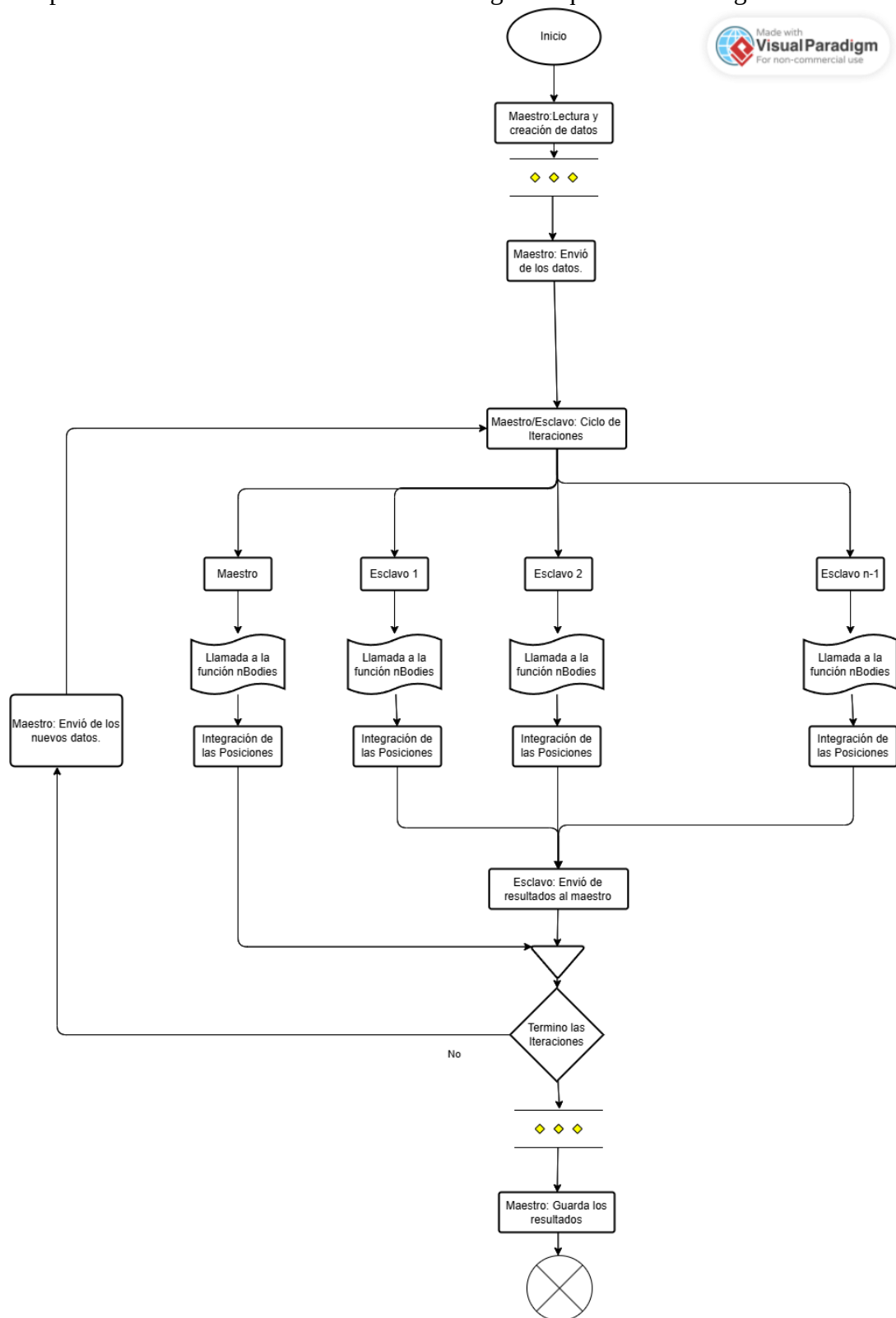
- Encargado de la entrada y salida del sistema.
- Encargado de la distribución de los datos a los esclavos.
- Realiza la división del trabajo a los esclavos.
- Se queda con un subconjunto del trabajo para el realizarlo.
- Realiza las medidas del tiempo.

Para los **Esclavos**:

- Reciben los datos entregados por el maestro.
- Reciben el subconjunto en el cual trabajaran.
- Realizaran los cálculos.
- Entregarán los resultados al maestro.

En esta definición de función el maestro no solo es el encargado de la gestión, sino que también en se va a quedar con un subconjunto de las tareas y trabajara, esto es para evitar que el maestro quede sin uso durante este periodo de tiempo.

Si aplicamos lo antes mencionado nuestro diagrama quedaría de la siguiente forma:



El caso que ilustre anteriormente es aplicando MPI, es decir, se distribuye la carga de trabajo entre varias máquinas. Pero también se puede distribuir la carga de trabajo dentro de cada máquina.

Paralelizando dentro de cada máquina.

Dentro de cada máquina podemos paralelizar el subconjunto de datos que nos llegan, esto mediante Openmp, lo que nos ofrece es que utilicemos todos los hilos disponibles en la máquina y esto con la finalidad de realizar la tarea con mayor velocidad.

Más, sin embargo, hay que tener en cuenta que no se puede abusar del uso tanto de Open como de MPI, esto debido a que las sumas no son de números enteros, sino de punto flotante lo que al utilizar más hilo y más procesos puede generar una diferencia muy grande respecto al valor real esperado.

Donde aplicar la Paralelización dentro de cada máquina.

Va a depender del problema a solucionar, en este caso y en la solución implementada se realizó en 2 lugares, el primero, dentro de la llamada al `nBodies`, esto debido a que `nBodies` dentro de su código posee un doble ciclo que lo convierte en orden n^2 entonces lo que busque allí es que cada hilo hiciera $(n/\#\text{hilos}) * n$ de esta forma ese tiempo de n^2 se reduce considerablemente, para este primero hay que tener mucho cuidado donde colocar la creación de hilos, debido a que si se hace en el interno se van a crear y destruir hilos n veces, es decir, por cada iteración de ciclo exterior se va a crear y destruir los hilos, en cambio, si se realiza en el ciclo más externo solo se crea y se destruye una sola vez, pero como mencioné la solución va a depender del problema a tratar. El otro lugar donde le coloque la paralelización aplicando Openmp fue en la función integración de datos, esta también es una función que en su código interno posee un ciclo de orden n , el cual con la paralelización se logra reducir a $n/\#\text{hilos}$ y este puede ser un comportamiento a seguir para saber dónde podemos aplicar la paralelización, el cual es: Buscar el código que tiene el mayor orden de toda la función e intentar su paralelización.

En conclusión.

Para finalizar, este código fue posible realizarlo aplicando tanto MPI como Openmp, pero hay que tener en cuenta que el resultado varía al original, no mucho pero no es igual. Tenemos que realizar un análisis a cada problema y sumar tanto los puntos positivos como los negativos para saber si es viable el uso de la paralelización del código, con ella se pueda ganar velocidad, pero puede ocurrir como en este caso que el resultado no es 100% fiable o incluso en otro caso el código resultado puede ser poco legible haciendo que su mantenimiento sea mucho más costoso.

Y tengamos en cuenta que la Ley de Amdahl nos recuerda que siempre habrá una porción de código que debe ejecutarse de manera secuencial, limitando el potencial de la paralelización.